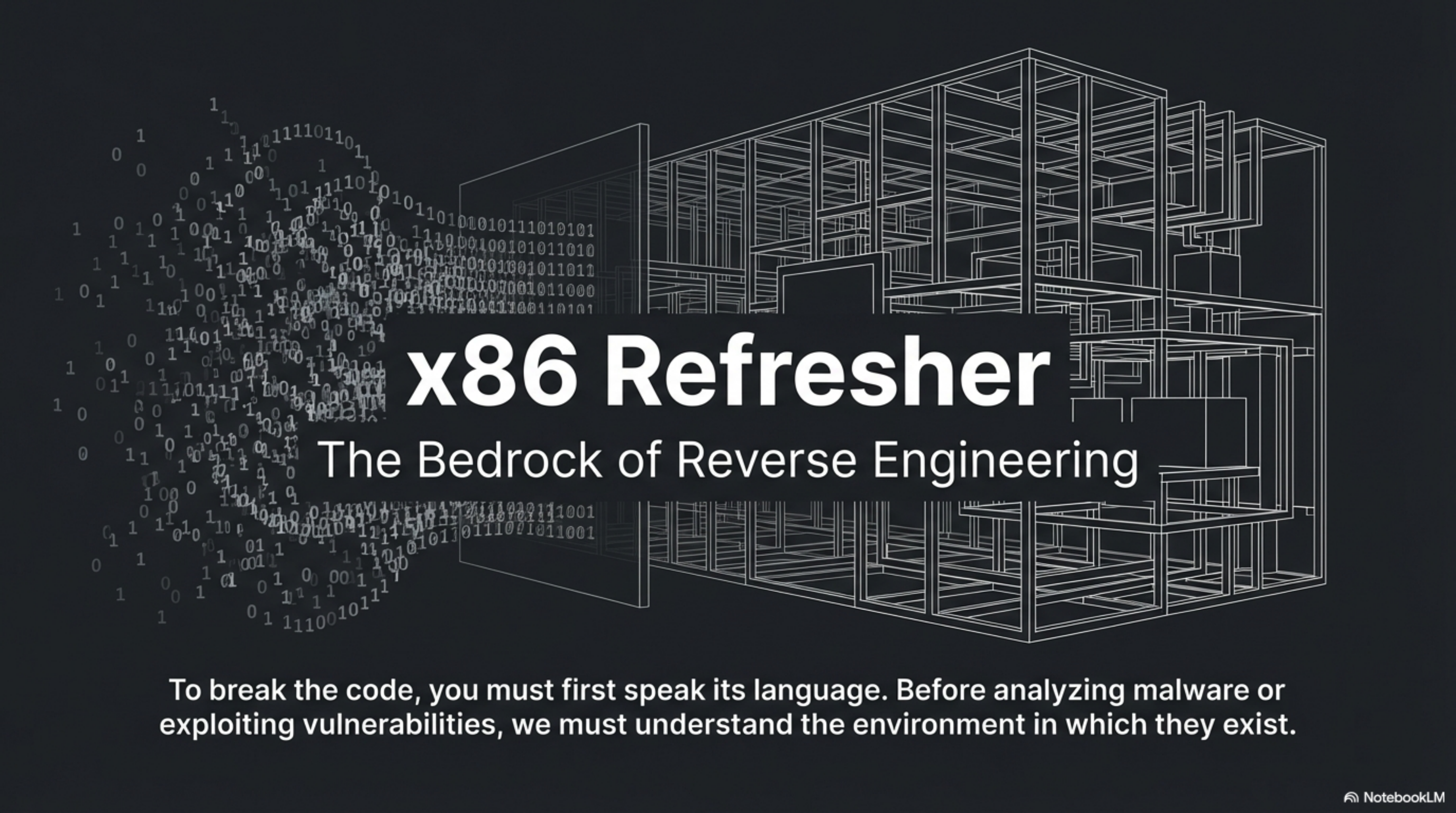


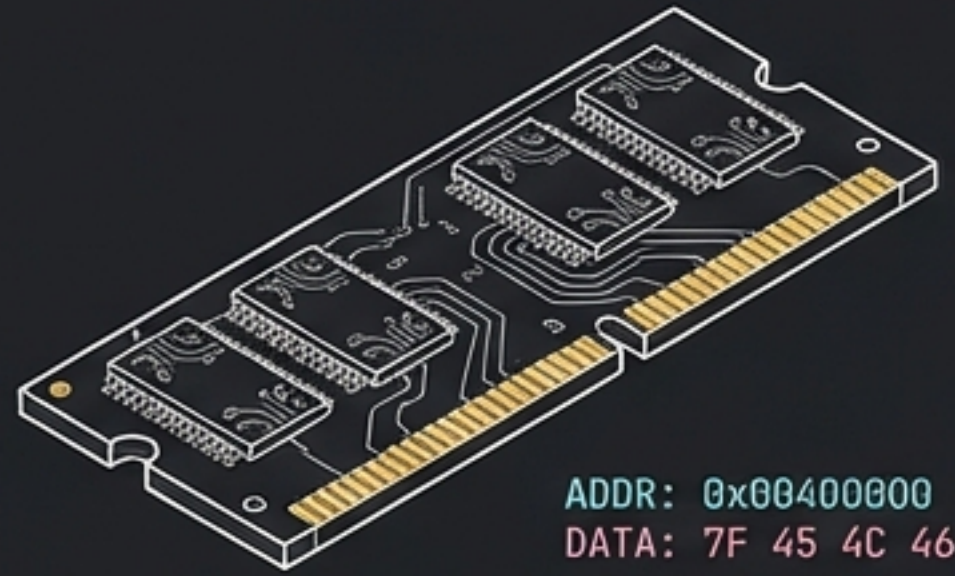


x86 Refresher

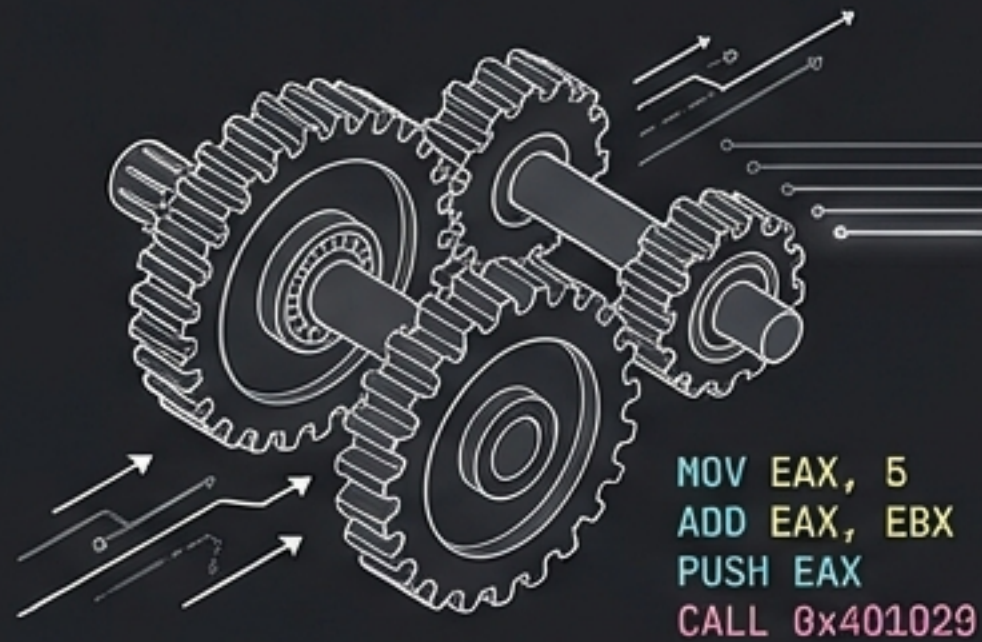
The Bedrock of Reverse Engineering

To break the code, you must first speak its language. Before analyzing malware or exploiting vulnerabilities, we must understand the environment in which they exist.

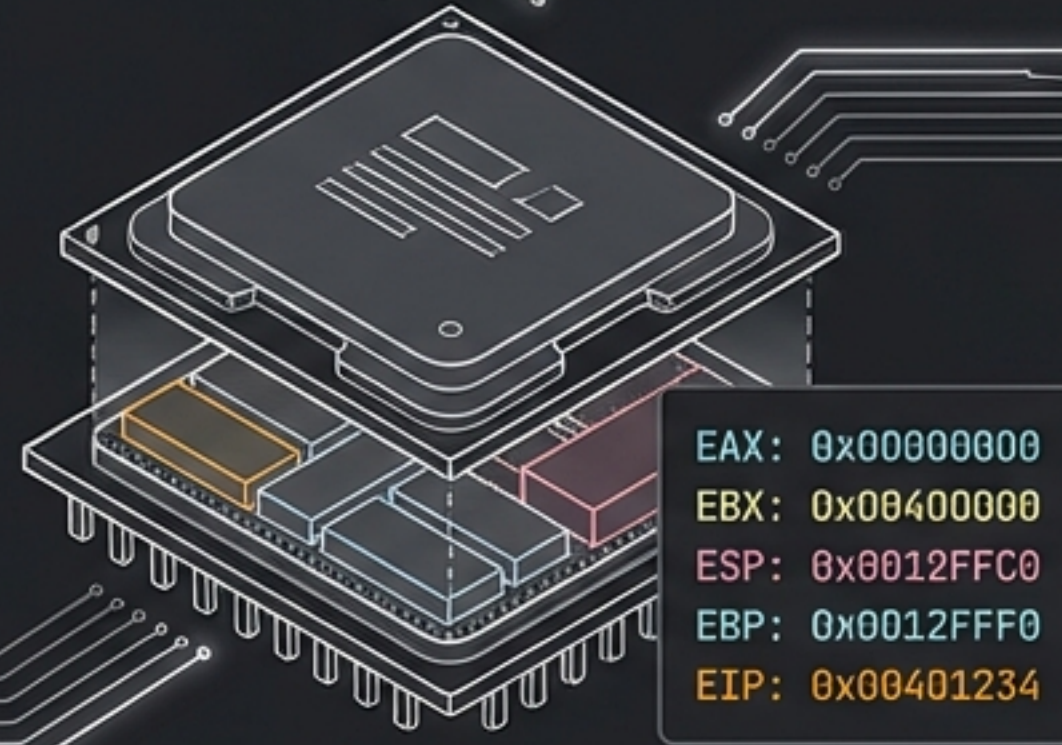
The Analyst's Toolkit



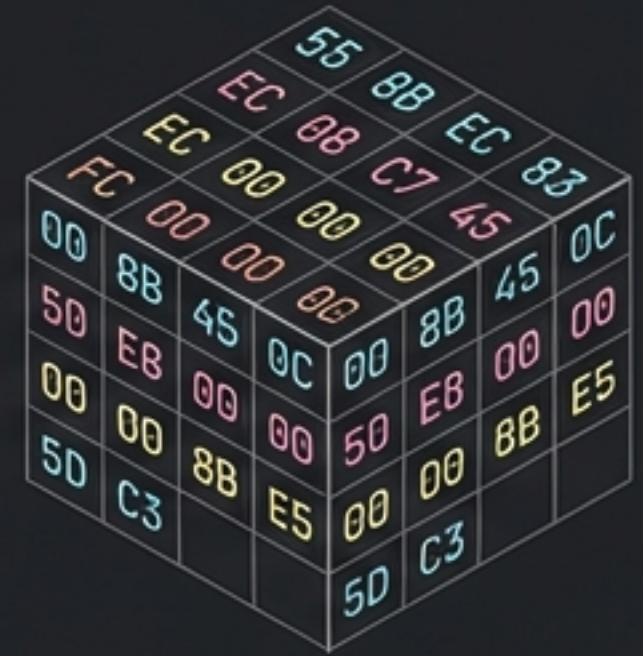
01. The Territory: Memory Management



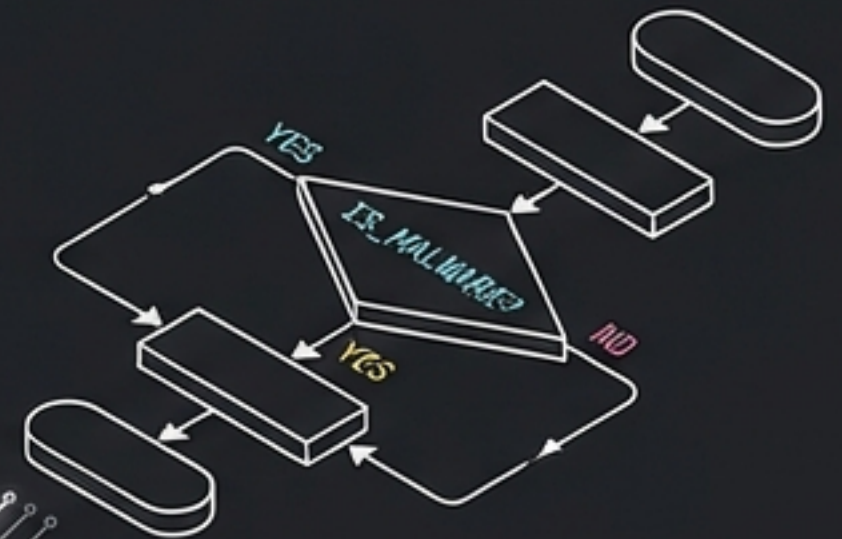
04. The Actions: Assembly Instructions



03. The Engine: Key x86 Registers



02. The Raw Material: Data Representation



05. The Process: Workflow & Analysis

32-bit Memory Management

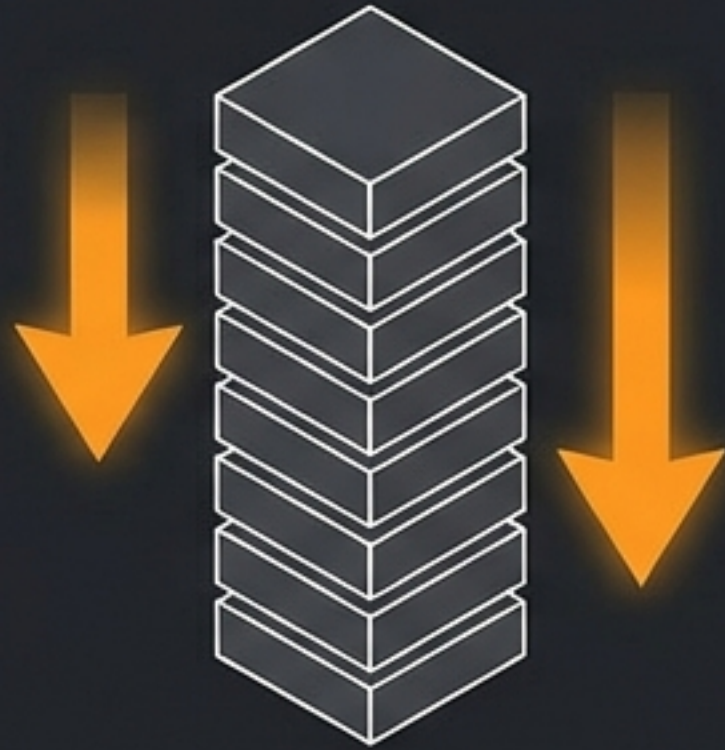


Virtual Address Space

- In a 32-bit environment, addresses are 32 bits (4 bytes) long.
- This creates a finite linear addressable space for the application.
- The OS maps this virtual space to physical hardware.

Structure vs. Capacity: The Stack and The Heap

THE STACK



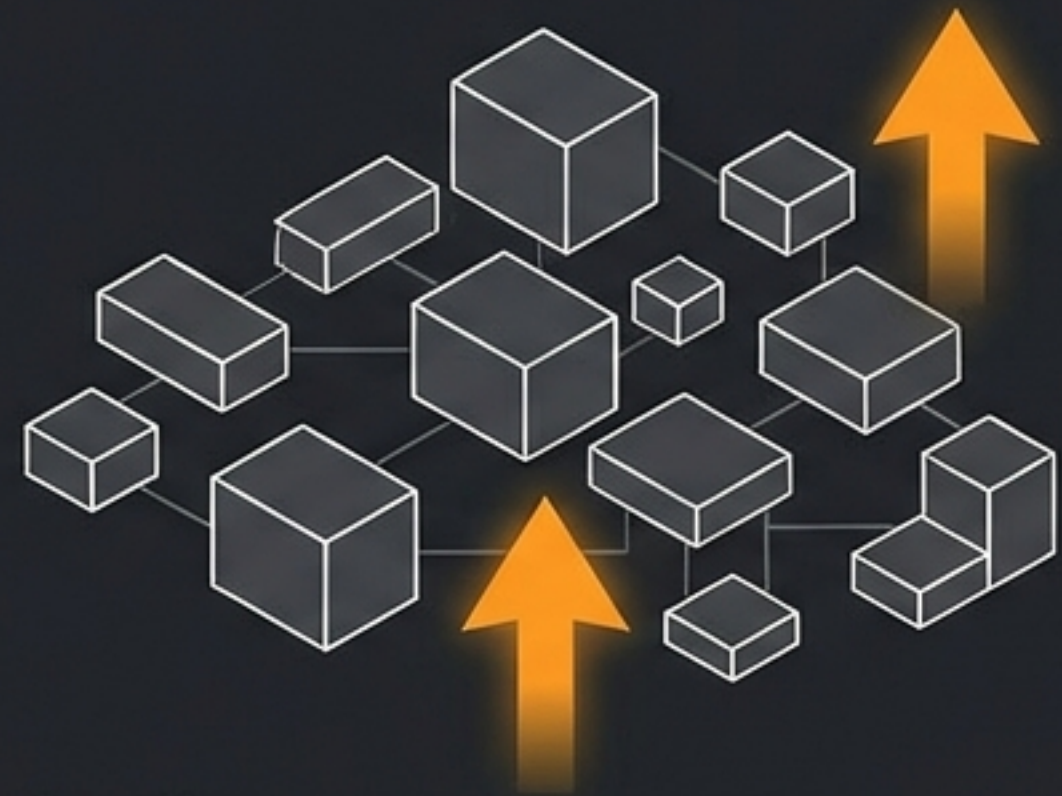
JetBrains Mono

LIFO (Last In, First Out)

Stores: Control Flow, Local Variables, Arguments

Grows: DOWN (High to Low Memory)

THE HEAP



JetBrains Mono

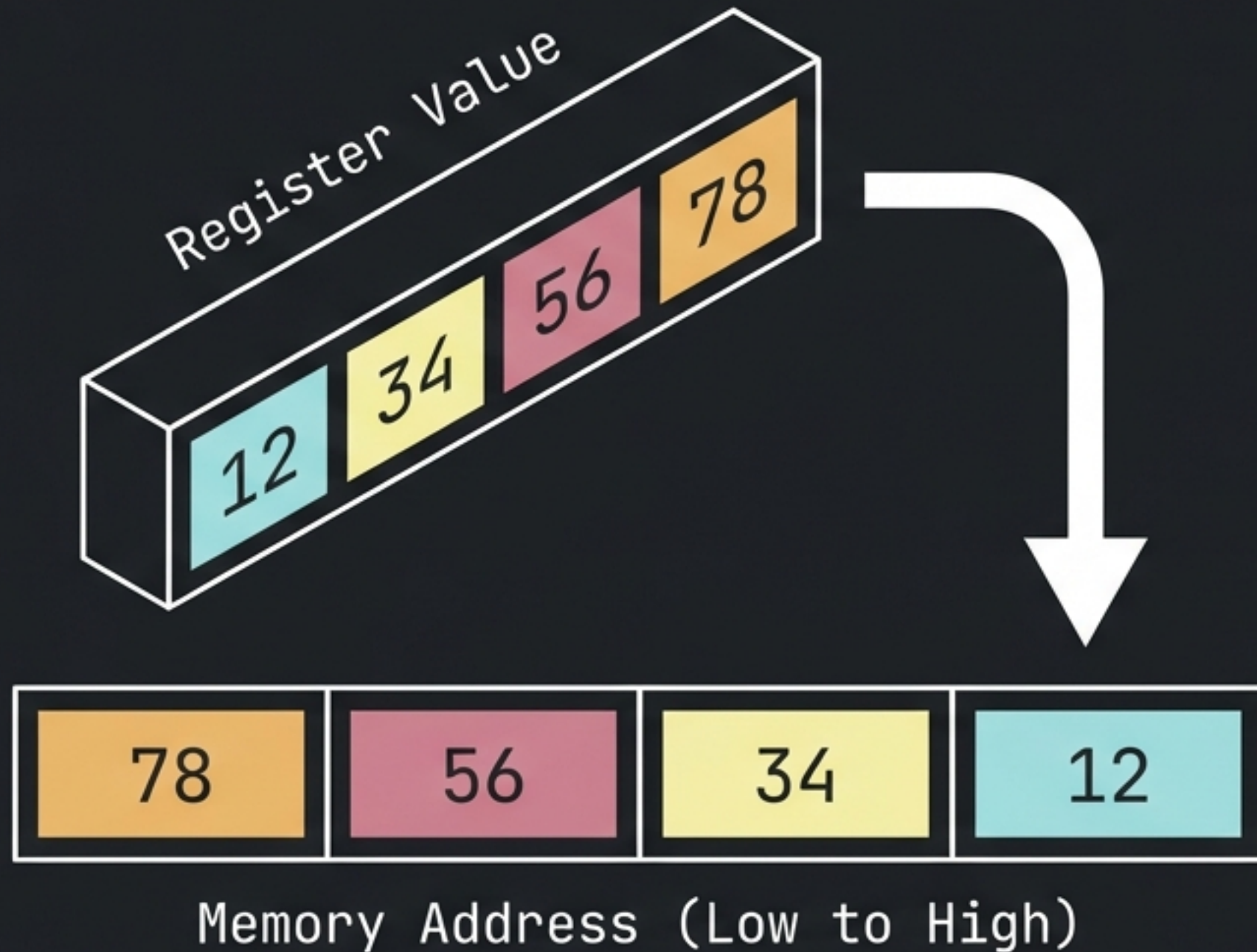
Dynamic Allocation

Stores: `malloc()` / `new` objects

Grows: UP (Low to High Memory)

The Stack is for structure; the Heap is for capacity.

Data Representation: Endianness



Little-Endian (x86 Architecture):

- Stores the least significant byte first.
- Mnemonic: Little End First.

Big-Endian (Network Traffic):

- Stores the most significant byte first.

The DNA of Execution: Opcodes

55

Hex / Opcode

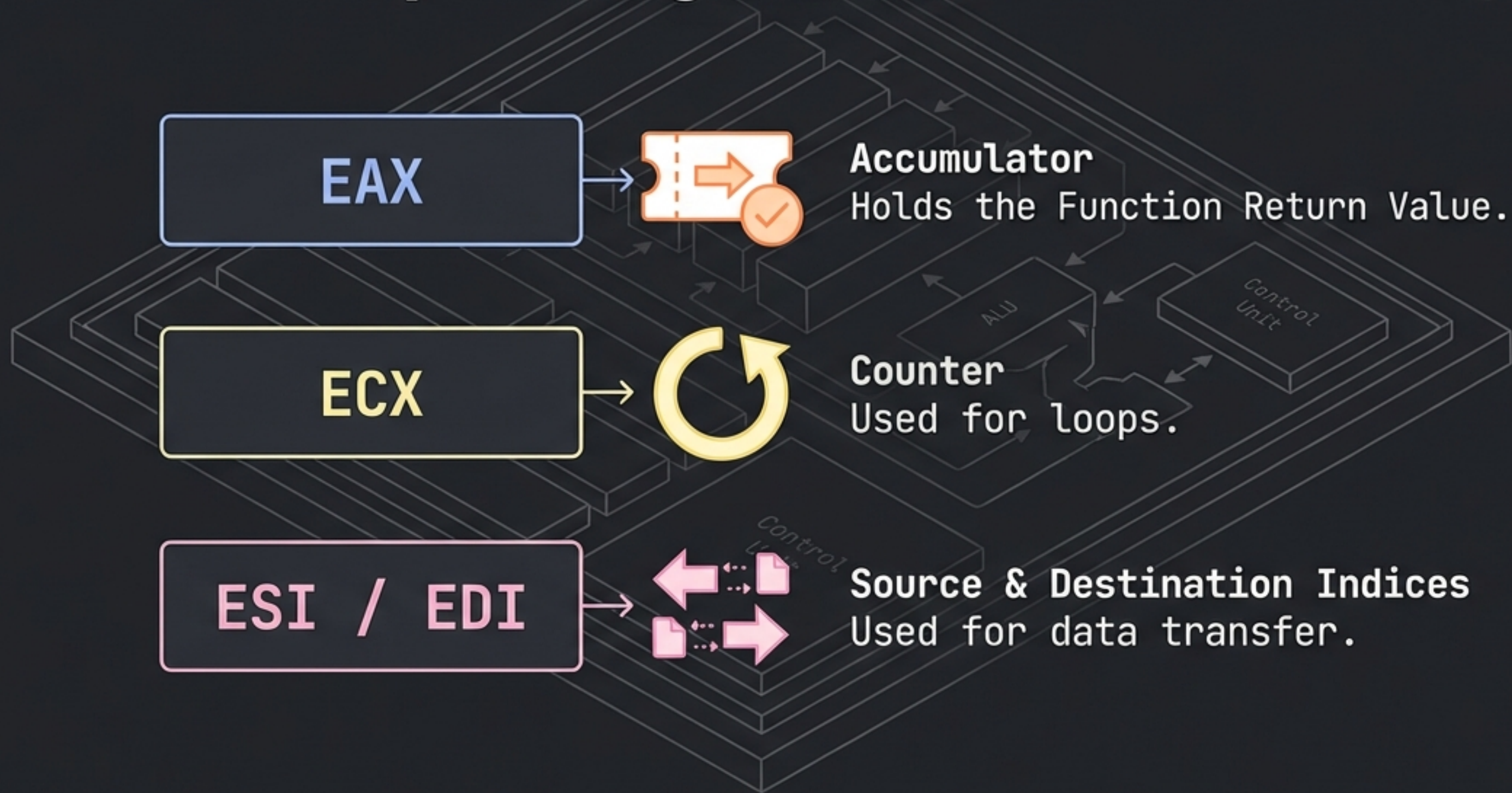


PUSH EBP

Assembly Mnemonic

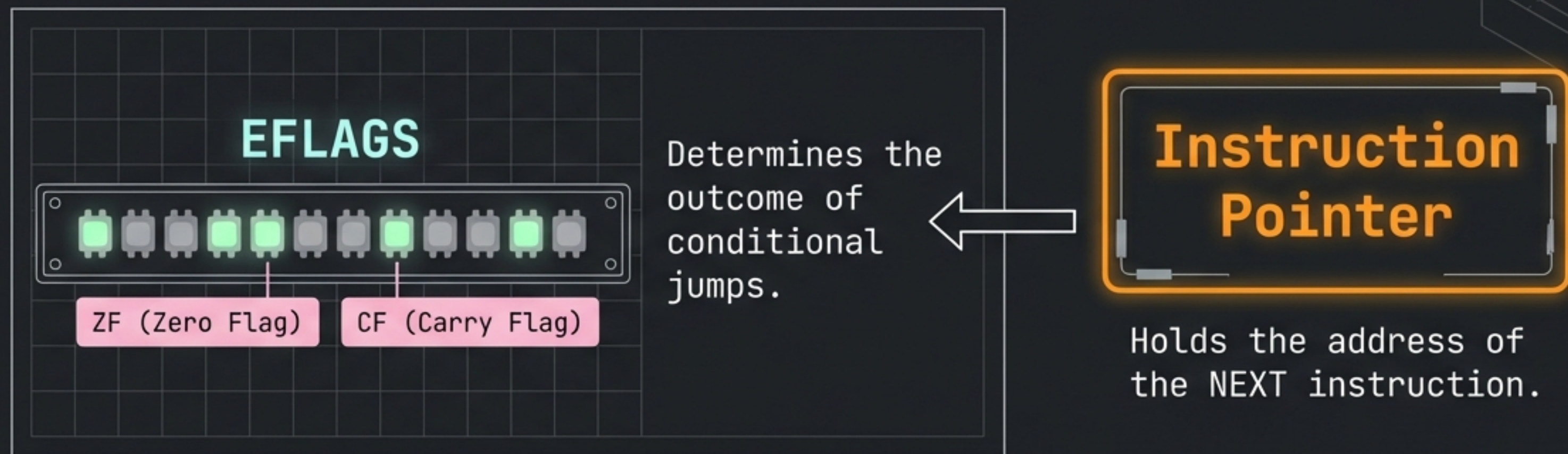
- Opcodes are the machine code representation of instructions.
- **Relevance:** Analysts read Assembly, but exploits and shellcode are written in Opcodes.
- This is the raw DNA of the program.

General Purpose Registers (The Workhorses)



Registers are the high-speed storage locations within the CPU.

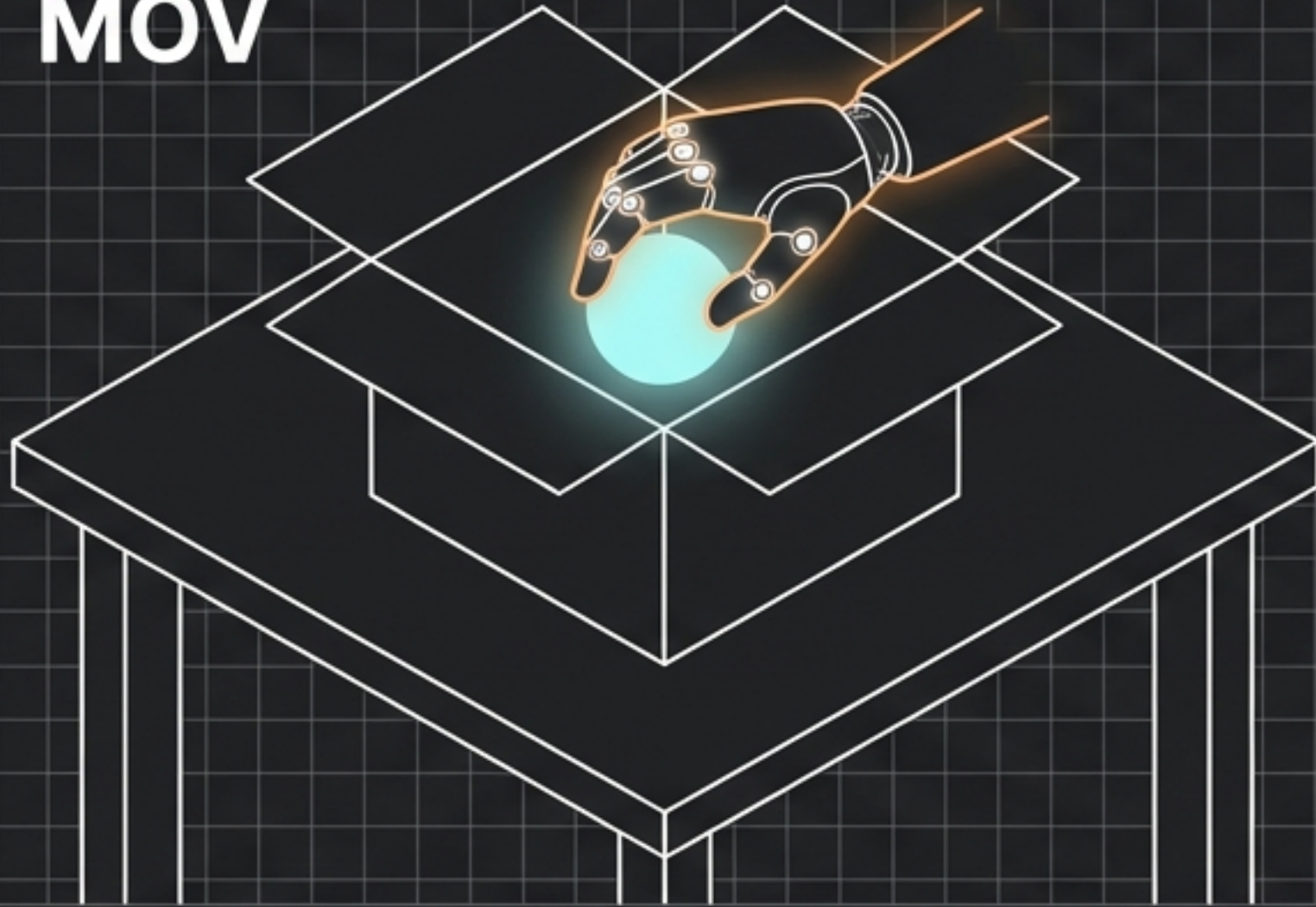
Control & Status Registers



Key Insight: The Holy Grail: Control EIP, and you control the execution. This is the bridge to exploitation.

Data Movement: Content vs. Location

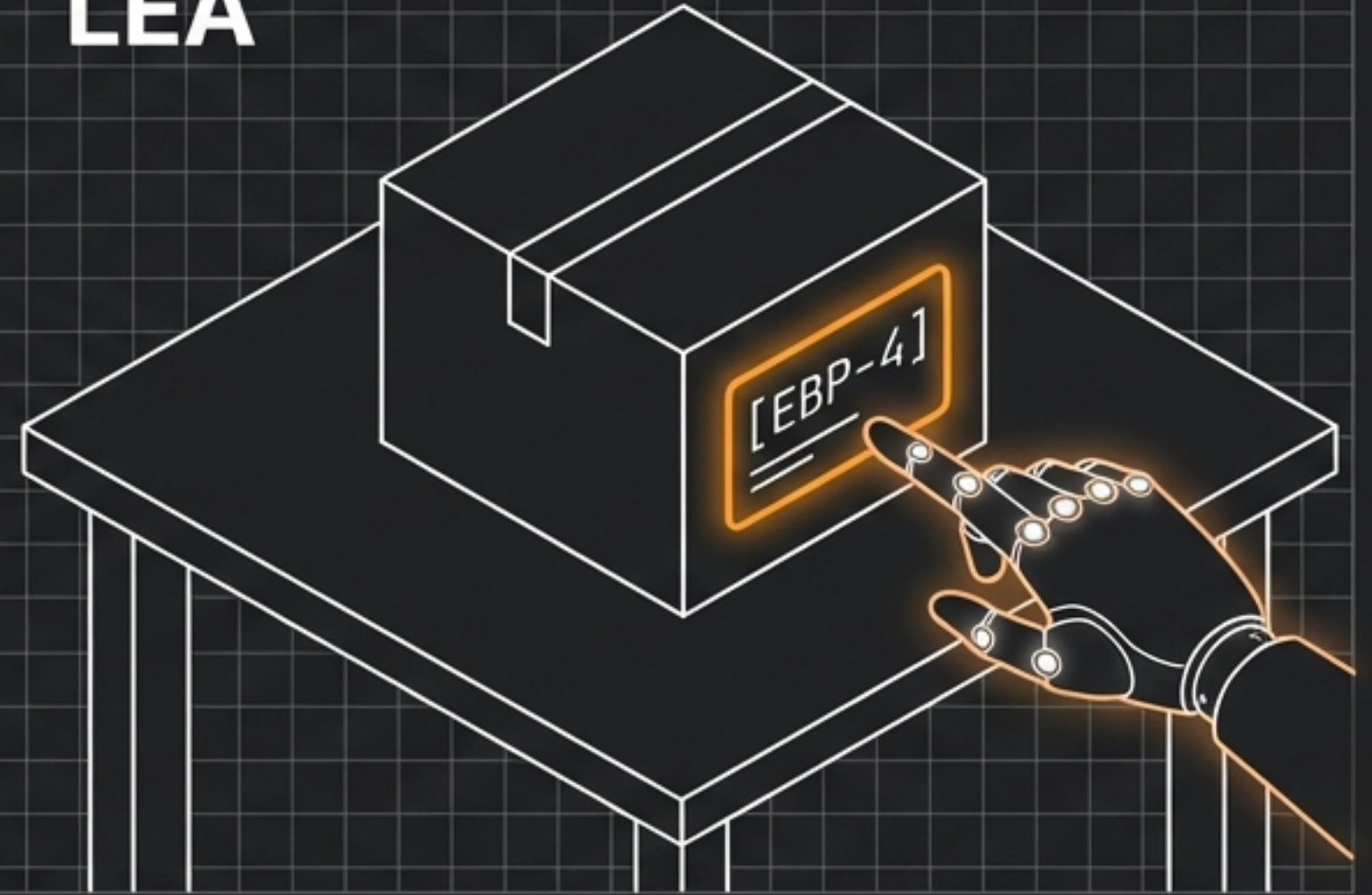
MOV



`MOV EAX, [EBP-4]`

Give me the CONTENT at the address.

LEA



`LEA EAX, [EBP-4]`

Give me the ADDRESS itself.

Arithmetic & Logic: The Zeroing Idiom -1

ADD - Addition



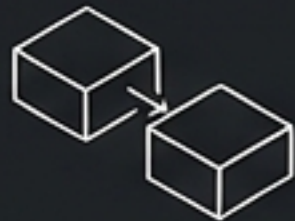
SUB - Subtraction



MUL - Multiplication



DIV - Division



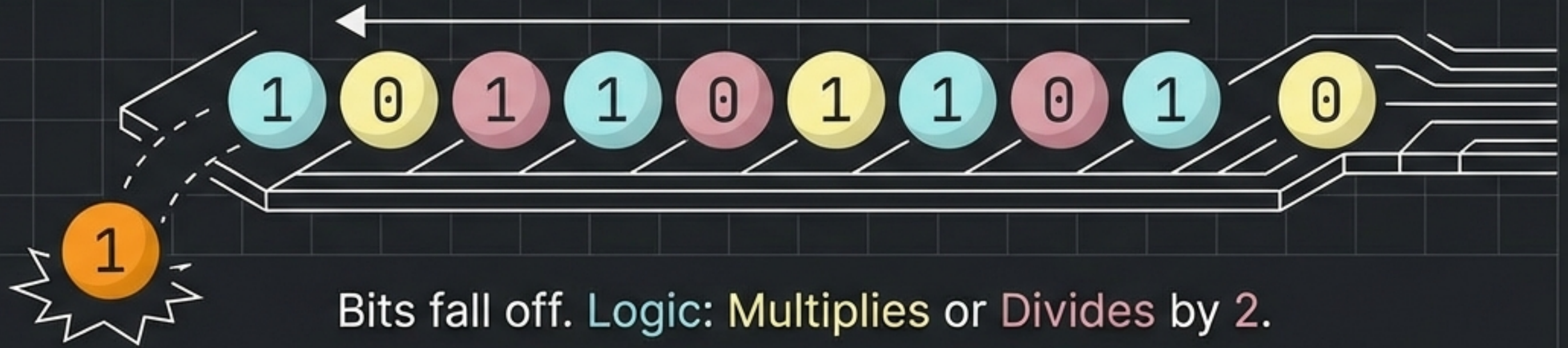
XOR EAX, EAX

The Analyst's Insight:
This sets EAX to 0.

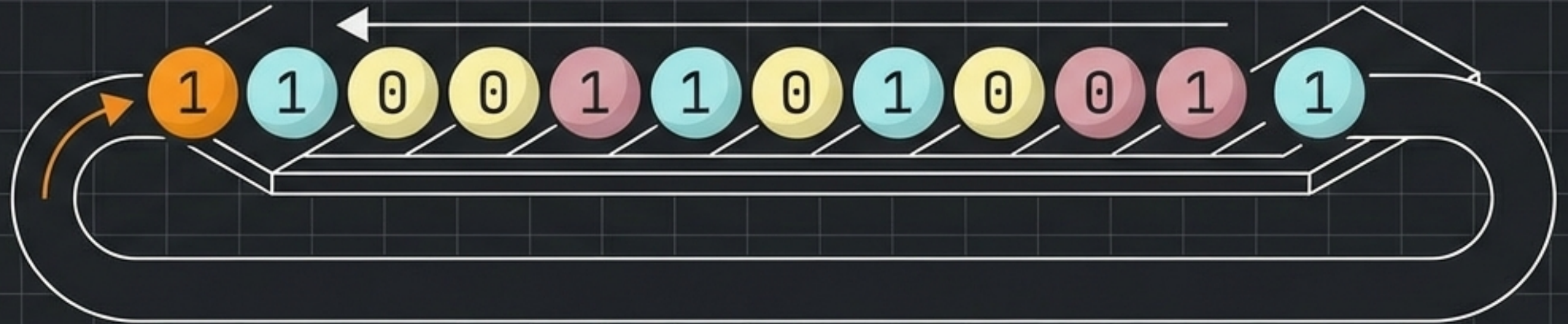
Why? Optimization. It is faster and occupies fewer bytes than 'MOV EAX, 0'. This is a standard compiler signature.

Bitwise Operations: Shifting & Rotating

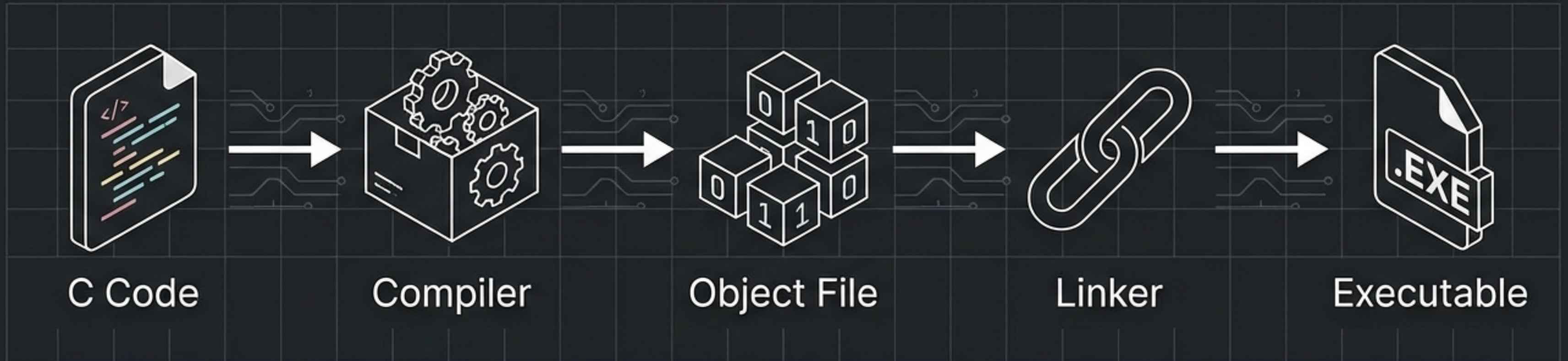
SHL / SHR
(Shift)



ROL / ROR
(Rotate)



The Workflow: From Source to Binary

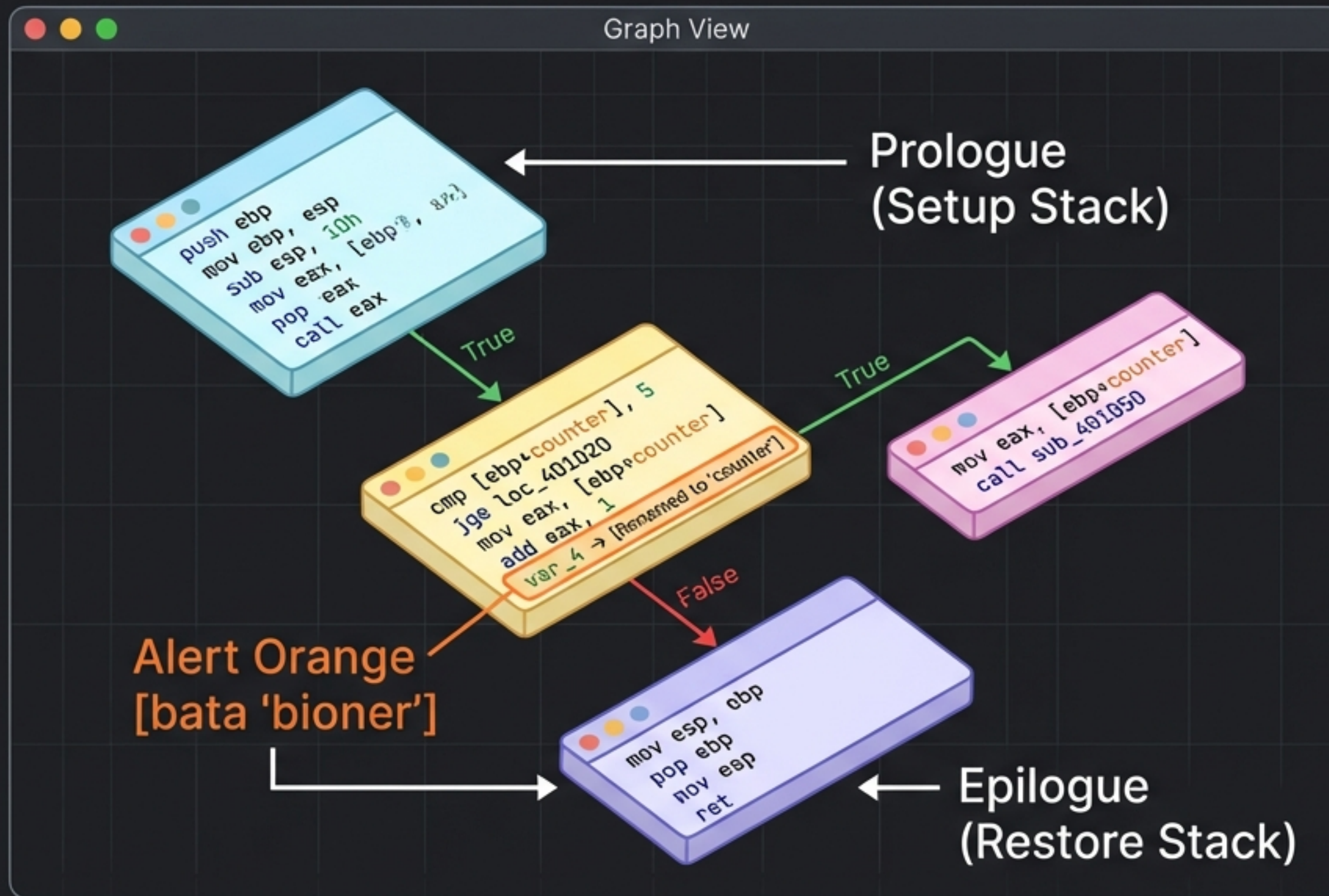


The Lossy Nature of Compilation

Source		Binary
<code>int x = 1 + 2;</code>		<code>MOV EAX, 3</code>

The logic (1+2) is gone. Only the result (3) remains. We analyze the artifacts, not the intent.

Navigation in IDA Pro



Key Techniques:

- Identify function boundaries via Prologue/Epilogue.
- Rename variables to reconstruct logic.
- Trace registers backwards to find data sources.

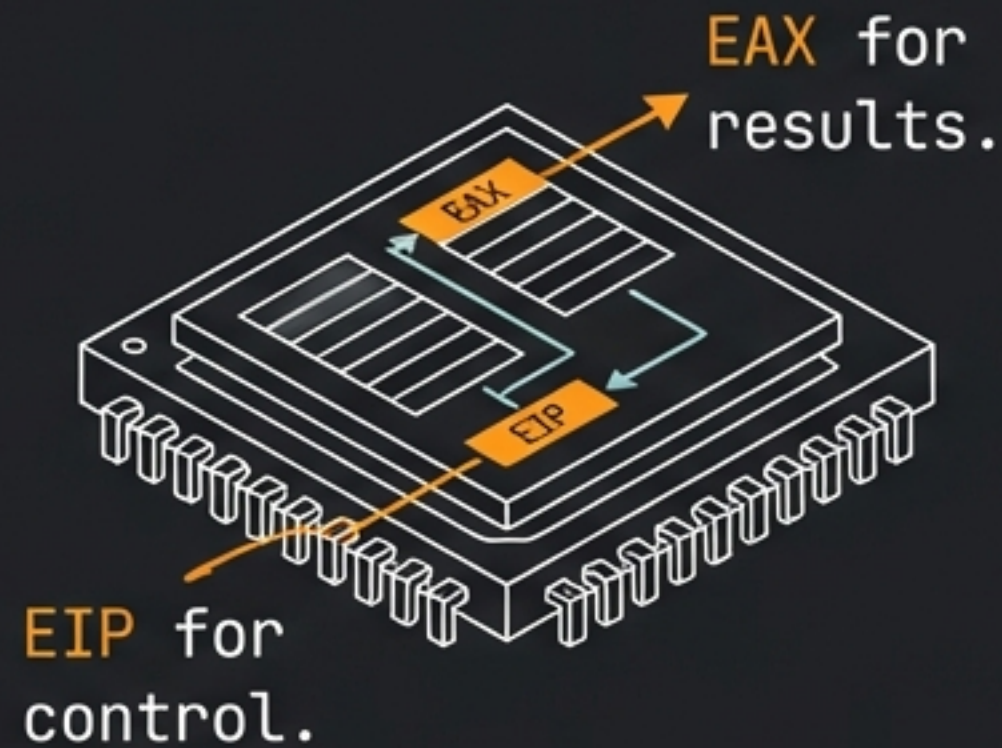
Summary: The Vocabulary of Analysis

Territory



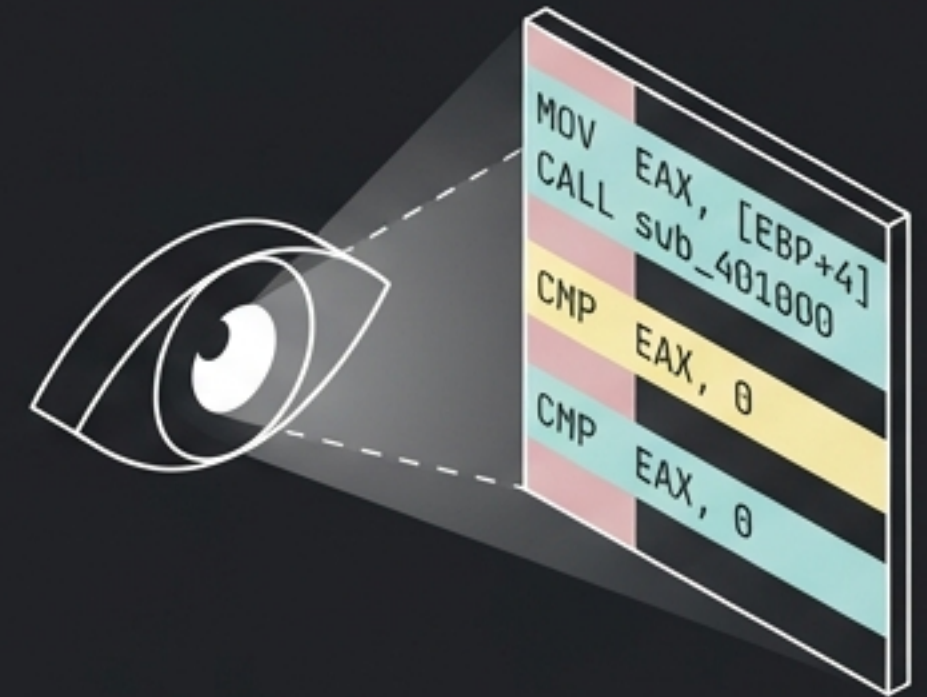
Stack grows down.
Heap grows up.

Engine



EAX for results.
EIP for control.

Process



Reconstructing high-level
logic from low-level
artifacts.

"Reverse Engineering is the art of finding the truth in the machine."

Knowledge Check

- | | |
|---|---------------------------|
| 1. Which register holds the return value? | 1. EAX |
| 2. Does the Stack grow up or down? | 2. Down |
| 3. MOV vs LEA: Which gets the address? | 3. LEA |
| 4. Why use XOR EAX, EAX? | 4. Optimization / Zeroing |
| 5. Which register is the loop counter? | 5. ECX |
| 6. 0x12345678 in Little-Endian? | 6. 78 56 34 12 |
| 7. Purpose of EIP? | 7. Next Instruction |